

A Single-Board Fermilab Timing Pipeline on the Xilinx KR260: Decode, Timestamp, Publish, Mirror

1. Introduction and motivation

A particle accelerator is a distributed machine whose thousands of devices, from magnet power supplies to beam instrumentation to the injection kickers, all have to act in step. They are kept in step by a family of *timing links*: serial broadcasts that carry short event codes (`beam is coming` , `supercycle starts now` , `reset this counter`) to every device at once. At Fermilab three of these links are relevant to this project, and they are three generations of the same idea:

- **TCLK** is the legacy link. It is a roughly 10 MHz biphasemark (Manchester) serial stream on a 3.3 V copper line. Each frame carries an 8-bit event code, written `$xx` in Fermilab notation, and nothing else. There is no timestamp on the wire: a device knows *what* happened and *that it just happened*, but the link itself does not say *when* in any absolute sense.
- **ACLK** is the newer gigabit link. It carries the same style of event, but as an 8b/10b framed packet on a 1.25 Gbps optical (SFP) transport, with room for a 16-bit event field, a 64-bit data payload, and a CRC-8. It runs far faster than TCLK, and the CRC-8 lets a receiver reject a corrupted frame.
- **ACLK-Lite** is the PIP-II down-converted variant: a Manchester stream meant to be carried on a simpler physical layer for the new linac, closer in spirit to TCLK than to full gigabit ACLK.

None of these links carries an absolute, disciplined notion of time on its own. That comes separately from **White Rabbit (WR)**, a sub-nanosecond timing distribution standard that delivers a 10 MHz reference plus a one-pulse-per-second (PPS) tick. WR is the source of *when*; the timing links are the source of *what*.

Fermilab timing links: how an event travels, and how time is kept

Events ride on TCLK and ACLK. Time rides on White Rabbit. This project decodes ACLK, stamps each event with WR time, and publishes it.

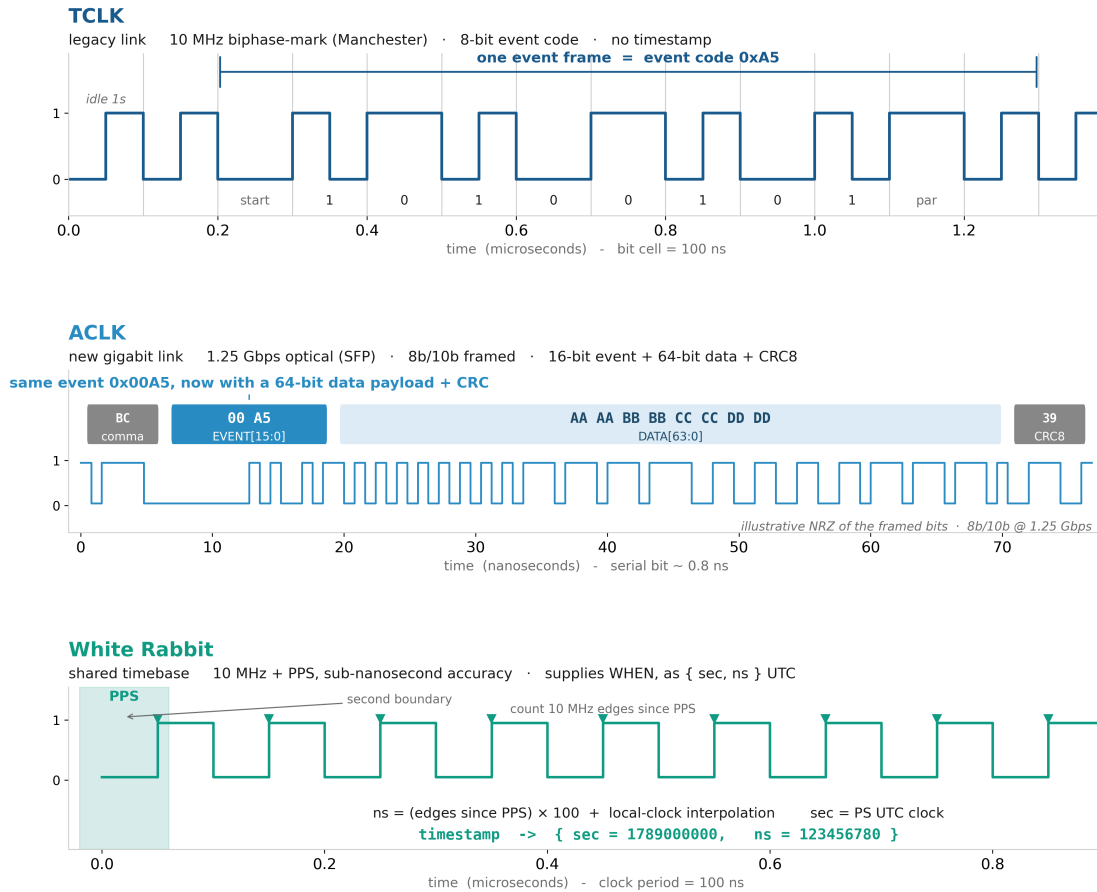


Figure 1. The three links. TCLK carries an 8-bit event code as a 10 MHz Manchester stream with no timestamp. ACLK carries the same event, now framed in 8b/10b at 1.25 Gbps with a 64-bit payload and a CRC-8. White Rabbit supplies the shared timebase: nanoseconds come from counting 10 MHz edges since the last PPS and interpolating, seconds come from the UTC clock, and the result is stamped onto every event as {sec, ns}.

The goal of this project is to build the piece that ties *what* to *when* and gets the result to software. Concretely: receive real Fermilab timing events on a single board, put an absolute {sec, ns} UTC timestamp on each event, deliver the timestamped stream to software over a standard interface, and, so that the whole thing can be exercised end to end without a second board, also re-broadcast the events back out over the accelerator's own timing-link transports. The deliverable is one FPGA bitstream plus the board-side software that reads it out.

2. System overview

The target is a **Xilinx Kria KR260** (Zynq UltraScale+, part xck26-sfvc784-2LV-c), a system-on-module that pairs a hardened ARM processing system (PS) running Linux with a large block of FPGA programmable logic (PL) on the same die, joined by AXI interconnect. The design is built around that split: the PL does everything that has to be fast and deterministic (recover bits from a wire, frame them, latch a

hardware timestamp the instant an event arrives), and the PS does everything that benefits from a general-purpose computer (poll the hardware, format records, talk to the network).

ACLK readout pipeline: decode, timestamp, publish, mirror

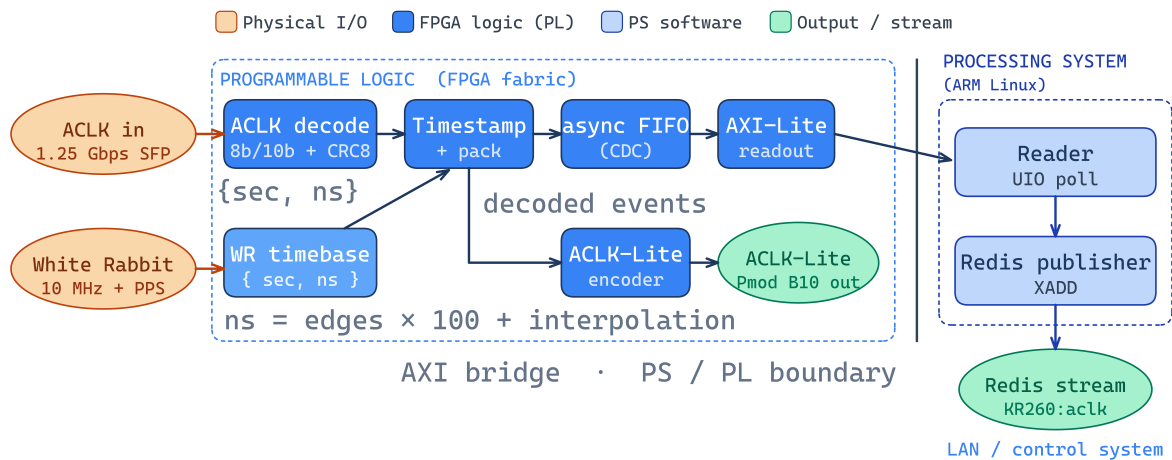


Figure 2. The end-to-end pipeline. In the PL: decode the link, latch a White-Rabbit timestamp, buffer across clock domains in an async FIFO, and expose the result over AXI4-Lite. In the PS: a reader polls the hardware and a publisher pushes each event into a Redis stream on the control network. The ACLK-Lite encoder mirrors the decoded event back out a Pmod pin as a scope probe.

What makes this a single-board loop rather than just a receiver is that the board also plays back what it receives. The signal path is:

```
TCLK (H12) + WR 10 MHz (E10) + WR PPS (E12)
-> decode TCLK, WR-timestamp each event
  -> AXI4-Lite readout -> PS reader -> Redis (the TCLK source)
  -> re-encode the events as ACLK frames
    -> GTH transmitter -> SFP+ --external fiber jumper--> GTH receiver
    -> decode ACLK, WR-timestamp on the same timeline
      -> AXI4-Lite readout -> PS reader -> Redis (the ACLK source)
      -> mirror as ACLK-Lite (Manchester) out a Pmod pin
```

Real TCLK comes in on a copper Pmod pin. The board decodes it, stamps it, and reads it out as one event source.

In parallel it re-encodes those same events into ACLK frames and transmits them out the SFP+ optical port through the FPGA's gigabit transceiver. A short fiber jumper loops that transmission straight back into the board's receiver, where it is decoded again, stamped again against the *same* White Rabbit timeline, and read out as a second, independent event source. Because both readouts share one timebase, a TCLK event and its looped-back ACLK twin carry directly comparable absolute times, which is exactly what you need to validate that the encode/transmit/receive/decode round trip is faithful. Finally the decoded ACLK events are mirrored a third time as an ACLK-Lite Manchester waveform on a Pmod output pin, available as a scope probe.

3. Physical I/O

The KR260 carrier card exposes its PL user I/O as four 12-pin Pmod connectors plus a Raspberry Pi header, all 3.3 V LVCMOS33 and wired to the FPGA through auto-direction level translators, so any pin can serve as an FPGA input or output. All of the low-speed link signals in this design land on **PMOD1**:

3.3V	GND	7	5	3	1
3.3V	GND	8	6	4	2

Figure 3. The Pmod connector layout. On PMOD1: pin 1 (package H12) is the TCLK biphase-mark input; pin 2 (B10) is the ACLK-Lite Manchester mirror output; pin 3 (E10) is the White Rabbit 10 MHz reference; pin 4 (E12) is the White Rabbit PPS. The two leftmost columns are the fixed 3.3 V and ground rails.

TCLK does not need a clock-capable FPGA pin. The decoder oversamples the line at 80 MHz and treats it as ordinary data (Section 4.1), so any general-purpose LVCMOS33 user pin works; a 10 MHz biphase-mark stream has edges no closer than about 50 ns apart, comfortably inside the translator's bandwidth. The gigabit ACLK link is the exception: it does not use a Pmod pin at all but the dedicated SFP+ cage, driven by the FPGA's gigabit transceiver (GTH) differential pairs and a 156.25 MHz reference clock.

4. The signal chain in technical depth

4.1 TCLK decode

Biphase-mark coding (the "mark" form of Manchester) puts a transition at the start of every bit cell and adds a second transition in the middle of the cell only for a one. The clock is therefore embedded in the data, which is robust, but it means the receiver has to recover bit boundaries from edge timing rather than from a separate clock line. The design does this by brute-force oversampling: `serdec4_9MHz` samples the raw line at 80 MHz (eight samples per 100 ns bit cell) into a shift register and recovers the biphase-mark cells from the sampled edge pattern. Downstream, `TCLK_DESERIALIZER2` and `TCLK_RCV` assemble the recovered cells into frames and pull out the 8-bit event code. The whole TCLK front end plus its readout and timestamp lives in `tclk_readout_top`.

The sampler runs faster than the line rate instead of at it because the bit boundaries land at an arbitrary phase relative to the FPGA's own clock. An oversampled, edge-detected recovery tolerates that phase far better than a sampler clocked at the nominal line frequency would.

4.2 The White Rabbit timebase

`wr_timebase` turns the White Rabbit 10 MHz reference and PPS into a free-running `{sec, ns}` counter. The nanosecond field is built by counting 10 MHz edges since the most recent PPS edge and multiplying by 100 ns per edge, with local-clock interpolation between edges for finer resolution; the seconds field comes from the PS UTC clock latched at the second boundary. The PPS edge resets the nanosecond count every second, which keeps it disciplined to the White Rabbit master rather than free-running off the FPGA oscillator.

The block enforces **strict validity**. If an event is stamped while the board is not yet White-Rabbit-synchronized, the timestamp is emitted as zero rather than as a plausible-looking but meaningless number, and software surfaces that to the operator as `UNSYNC`. This is a safety property: a wrong timestamp that looks right is worse than an obviously invalid one, because a physicist correlating events after the fact has no way to tell a subtly wrong time from a correct one.

Both event readouts are stamped from `wr_timebase`, instantiated as two replicas that share the same reference so the TCLK and ACLK paths sit on one timeline. A third register slave, `wr_timebase_axi`, exposes the timebase to software for monitoring and arming.

4.3 The readout core and AXI4-Lite interface

Both the TCLK and the ACLK readouts are built from the same two modules, which is what keeps the two paths behaving identically. `aclk_readout_core` latches a 64-bit hardware timestamp the instant an event is presented, packs the event through a null-drop stage (events that decode to a null code are discarded in hardware so they never reach software), and carries the packed record across the clock-domain boundary through a dual-clock `async_fifo`. Crossing that boundary safely matters because the decode logic and the AXI interface run on different clocks; the async FIFO with Gray-coded pointers is the standard, verified way to hand data between them without metastability.

`aclk_readout_axi` wraps that core in an AXI4-Lite register block that the PS reads. Software sees a small register file: status (FIFO empty / overflow), the event code and its flags, the 64-bit data payload split into high and low words, the 64-bit timestamp split into high and low words, a write-only POP register to advance the FIFO, running event / null / error counts, health registers (line activity, receive-clock heartbeat, MMCM and WR lock), and a 256-bit event drop-mask filter (`FILTER_CFG` to program which event codes to discard in hardware, `FILTERED_COUNT` to report how many were dropped).

One hardware-specific quirk shaped the register layout: on the KR260's low-power-domain AXI path, the hand-written AXI4-Lite slave only returns read data correctly at 16-byte-aligned offsets, so every register is spaced 16 bytes apart rather than packed at 4-byte word boundaries. The full field-level register map for both readouts and the WR monitor slave is in the generated hardware interface guide ([docs/generated/tclk-aclk-pipeline-hardware-interface-guide.pdf](#)); this report gives the shape rather than transcribing every bit.

4.4 TCLK to ACLK re-encode and the gigabit loop

To close the loop, decoded TCLK events are re-encoded as ACLK frames. `aclk_tclk_encoder` gearboxes each 8-bit TCLK event into the wider ACLK frame format (event field, data payload, and a CRC-8 computed

by `crc8_calc`), using the 16-to-96 and 96-to-16 bit gearboxes that match the transceiver's internal width to the frame width. The framed stream is 8b/10b encoded and driven out the SFP+ by `ac1kgt_gt`, the Xilinx gigabit transceiver (GTH) wizard IP configured for 1.25 Gbps with a 156.25 MHz reference clock, committed to the repository as a generated `.xci`.

An external fiber jumper loops the transmitter's optical output straight back into the same transceiver's receiver. On the receive side `ACLK_RCV` and the matching gearboxes and CRC check decode the frame back into events, which `ac1k_gt_readout_top` stamps against the shared White Rabbit timeline and reads out through the second `ac1k_readout_axi` instance. This is the mechanism that lets one board exercise the full encode / serialize / optical-link / deserialize / decode chain: the ACLK source in the data is literally the board's own TCLK events after a complete round trip through the gigabit link.

4.5 The ACLK-Lite mirror

The last stage takes the decoded ACLK events and mirrors them out as an ACLK-Lite Manchester waveform. `ac1k_lite_bridge` adapts the decoded event interface to the encoder, and `ac1k_lite_encoder` drives the biphasemark output on PMOD1 pin 2 (B10). This output is a scope probe: it makes the down-converted PIP-II-style link observable on a benchtop oscilloscope and provides a physical ACLK-Lite source for testing downstream receivers, all without any additional hardware. The exact on-wire framing for ACLK-Lite (per-byte parity, byte-oriented cells, the terminal-cell convention, and the 1/2/12-byte frame lengths) is documented authoritatively in `docs/ac1k-lite-framing.md`.

5. Software readout path

On the PS side the design is deliberately thin, because the hard real-time work has already been done in the PL. Two small Python readers, one per source, map the AXI4-Lite register block through Linux UIO and poll it: read the status register, and while the FIFO is non-empty, read the event, its flags, its data payload, and its 64-bit timestamp, then write the POP register to advance to the next record. Reading through UIO rather than a custom kernel driver keeps the board-side software simple and portable.

Each reader hands its events to a Redis publisher, which pushes them into a Redis stream on the control network with `XADD`. The stream key follows a namespaced convention (for example `KR260:ac1k`) rather than a link-specific name, so multiple boards and sources coexist cleanly. Each event is written with a Redis stream ID derived from its White Rabbit event time rather than from Redis's own arrival time, with a monotonic guard to keep IDs strictly increasing; per-event-code index hashes, a status key, and a watchdog round out the schema. Persistence is turned off in the Redis configuration because this is a live telemetry stream, not a database of record, and the stream-ID-from-event-time convention requires Redis 7.0 or newer for the millisecond-plus-sequence ID syntax. A downstream consumer anywhere on the network then sees timestamped accelerator events in event-time order, keyed so they can be sliced by event code.

6. Verification and results

Development followed a simulation-first discipline. The inner loop is a cocotb (Python) testbench suite run under Icarus Verilog, with one testbench per module and an end-to-end chain testbench

(`aclk_pipeline_chain`) that exercises the whole pipeline in simulation before any bitstream is built. Every stage (TCLK receive, ACLK receive, the readout and its AXI register map, the async FIFO, the encoders, the White Rabbit timebase) has its own testbench, and each emits a matplotlib plot on completion. The board-side Python has its own `pytest` suite covering the register map, the Redis publisher, and the statistics reconciliation.

The load-bearing hardware validation is a **15.6-hour continuous dual-source capture** run on 2026-07-16. Over that run the board decoded real Fermilab TCLK, looped every event through the ACLK gigabit fiber path, stamped both sources against the shared White Rabbit timeline, and published both streams into Redis, accumulating roughly **5.55 million events per source with zero loss** (verified by reconciling the hardware event counters against the number of records that reached Redis and the on-disk statistics log). That single run exercises TCLK decode, the White Rabbit arm and lock, the gigabit SFP loop, and both readouts together under sustained real load.

Timing events published to Redis

Live Fermilab TCLK, White Rabbit timestamped, XADD to the `KR260:acLk` stream.

KR260:acLk = 5,556,662 events published · **captured over 15.6 h** · **~99 events/s**

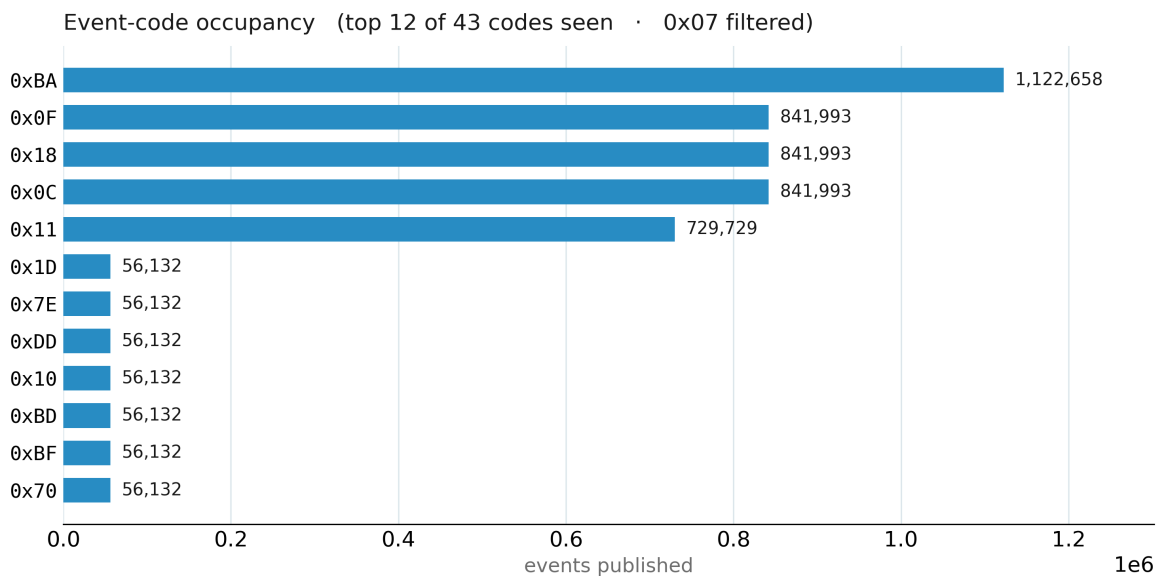


Figure 4. Event-code occupancy from the 15.6 h run: 5,556,662 events published to the `KR260:acLk` stream at about 99 events per second, resolved into the 43 distinct event codes seen (top 12 shown), with code `0x07` demonstrating the hardware drop-mask filter in action. The strongly non-uniform distribution is the real structure of the Fermilab timing broadcast, not an artifact of the readout.

Because every event carries an absolute timestamp, the captured stream can be analyzed the way a physicist would analyze real accelerator data. Folding the timestamped events on the 60-second accelerator supercycle recovers the periodic structure of the machine directly from the published Redis stream:

ACLK events folded on the 60 s supercycle

150 supercycles folded on \$00, median length 59.993 s, window 0-5 s

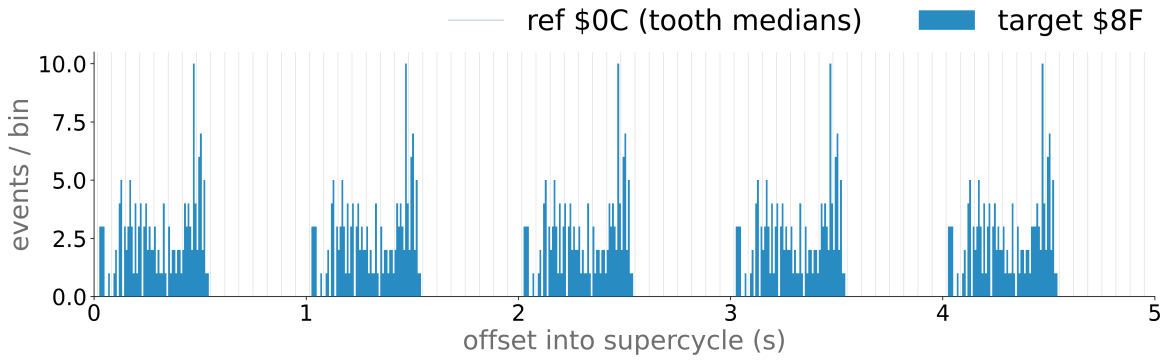


Figure 5. Timestamped events folded on the 60 s supercycle (histogram). 150 supercycles are overlaid on event code \$00 ; the repeating structure across the window is the machine's own periodicity, reconstructed purely from White-Rabbit timestamps in the captured stream.

ACLK events folded on the 60 s supercycle

150 supercycles folded on \$00

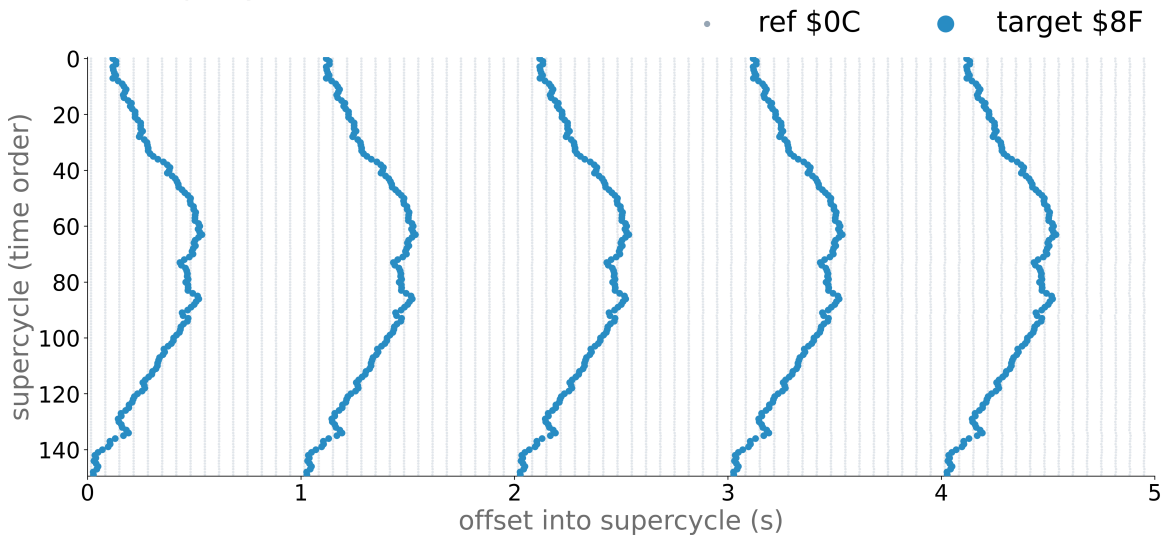


Figure 6. The same fold as a raster, one row per supercycle in time order. The stable vertical bands confirm that events land at consistent phases of the supercycle across the whole capture, which is the timestamp chain doing its job end to end.

Not everything is closed. The ACLK path is today exercised only through the board's own re-encode and fiber loopback, not against a live upstream accelerator ACLK feed, so confirming the ACLK CRC-8 polynomial against a real ACLK source remains the main deferred item. The White Rabbit PPS also needs a sufficiently wide pulse to be seen by the 40 MHz sampler; a real 10 ns PPS is too narrow and must be widened upstream or stretched in RTL.

7. Repository and how to build and run it

The repository is one product: the `aclk_pipeline` bitstream plus its board-side software. There is a single build target. `hw.ps1 build` (or `hw.sh build`) defaults to `vivado/build_aclk_pipeline.tcl`, which sources the RTL, runs Vivado in batch, and packages the result with `bootgen`. For historical reasons the block design keeps the internal name `uart_echo_bd`, so the output file is `uart_echo_bd_wrapper.bit.bin`; the name is cosmetic and keeps the board's overlay and UIO identity stable across builds. The build derives its 80 MHz and 40 MHz event-domain clocks and a 50 MHz transceiver free-run clock from `p1_clk0` with clock-wizard MMCMs, because a runtime `fpgautil` bitstream load does not reprogram the PS PL clock frequencies. It also uses an AXI SmartConnect on the LPD master, since the automatic interconnect corrupts the AXI4-to-AXI4-Lite read path on this hardware.

The RTL is organized by function:

Area	Files	Role
TCLK front end	<code>serdec4_9MHz.v</code> , <code>TCLK_DESERIALIZER2.v</code> , <code>TCLK_RCV.v</code> , <code>tclk_readout_top.sv</code>	recover and frame TCLK, timestamp, read out
ACLK front end	<code>ACLK_REV.v</code> (<code>ACLK_RCV</code>), <code>gearboxes</code> , <code>crc8_calc.v</code> , <code>aclk_gt_readout_top.sv</code> , <code>aclk_tclk_encoder.v</code>	encode/decode ACLK over the GT, timestamp, read out
Shared readout	<code>aclk_readout_core.sv</code> , <code>aclk_readout_axi.sv</code>	timestamp latch + null-drop + async FIFO + AXI4-Lite face
Timebase	<code>wr_timebase.sv</code> , <code>wr_timebase_axi.sv</code>	shared White Rabbit {sec, ns} + monitor slave
ACLK-Lite out	<code>aclk_lite_bridge.v</code> , <code>aclk_lite_encoder.sv</code>	mirror decoded events as Manchester on B10
CDC + top	<code>synchronizer.sv</code> , <code>async_fifo.sv</code> , <code>cdc_gray_count.sv</code> , <code>aclk_pipeline_bd_top.v</code>	clock-domain primitives and the integrated block-design top
GT IP	<code>vivado/ip/aclkgt_gt/aclkgt_gt.xci</code>	gigabit transceiver wizard IP (GTH, 1.25 Gbps, 156.25 MHz refclk)

The three AXI4-Lite slaves (the two readouts and the WR monitor) are inferred from `aclk_pipeline_bd_top` and fanned out from the PS master by a single SmartConnect. Simulation uses the `sim.ps1` / `sim.sh` wrappers (`sim.ps1 run -Module aclk_pipeline_chain` runs the end-to-end chain); the bitstream build and board deploy use the `hw.ps1` / `hw.sh` wrappers. For the operator's step-by-step procedure (load the bitstream, wire the board, run a capture, get the data out) see `docs/OPERATIONS.md`, and for the exhaustive register-level interface see the generated hardware interface guide under `docs/generated/`.